Technical Requirements Document

Stitch: the tools, technologies and infrastructure the product is built on.

1 Overview

This document outlines the tools, technologies and infrastructure used to build Stitch. It keeps everyone aligned on the stack, the move to a proper domain and host, and what the Play Store and App Store require.

2 Technical Goals

Secure
Row-level security, server-side money rules

Fast
Good on cheap phones and slow networks

Maintainable
Easy changes as the team grows

Ready to grow
More campuses, countries and users

3 Technology Stack

Area	Tool / Technology	Purpose
Frontend	HTML, CSS, JavaScript (no framework or build step today)	Builds the interface; later move to Vite and TypeScript
Mobile apps	Capacitor wrapper	Publishes the same app on Play Store and App Store with native features
Backend logic	Supabase Edge Functions (20)	Payments, AI, email, push, verification, admin actions
Database	Supabase Postgres 17 (London)	All app data; row-level security on every table
Authentication	Supabase Auth	Email code, Google, Apple sign-in
File storage	Supabase Storage (17 buckets)	Photos, media, voice notes, documents, CVs, verification files
Realtime	Supabase Realtime	Live chat, notifications, presence
Payments	Paystack	Card and Mobile Money collections and payouts
Identity checks	Smile ID	ID and selfie verification for posters
AI	AI model via ai-proxy and career-ai	Stitch Bot and Match with CV
Push	Web push now; Firebase and Apple push for apps	Alerts to users
Calls	WebRTC, with owned relay (TURN) to add	Voice and video calls
Group calls	Managed media server (such as LiveKit) to add	Class meetings, lectures, group calls
Hosting (target)	Cloudflare Pages	Serves the web app; replaces Vercel
Domain and DNS	Own domain on Cloudflare DNS	Brand address, subdomains, email setup
Analytics	PostHog	Usage, funnels, retention

Area	Tool / Technology	Purpose
Error tracking	Sentry	Crashes and errors
Fonts	Montserrat (main), Colmeak, Inter	Typography; Colmeak already embedded in the app code
Offline and install	Service worker now; web manifest to add	Offline page and installable web app

4 System Architecture

The app is a set of static files. All data and logic live in Supabase and a few outside services.

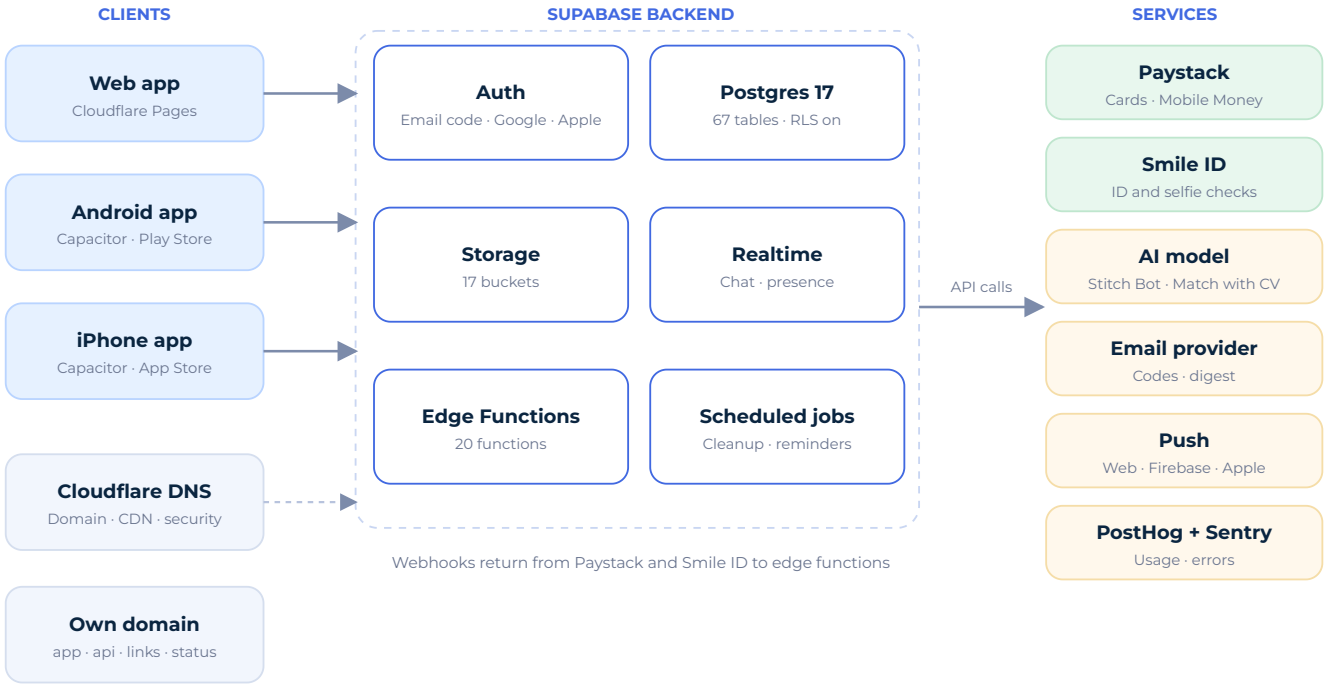


Figure 1: How the app, the backend and outside services connect.

5 Environments

Development
For building. Fake data only.

Staging
Copy of production for testing before release. Fake or cleaned data only.

Production
Real users only. Changes arrive only after passing staging.

- Each environment has its own Supabase project and its own keys
- Every database change is a migration saved in git
- Real user data is never copied into development or staging

6 Domain and Hosting

Moving from Vercel to a proper domain and host:

Migration path



Address	Use
yourdomain.com	Landing page
app.yourdomain.com	The web app
api.yourdomain.com	Custom domain for Supabase, so users never see supabase.co
links.yourdomain.com	Share and deep links
status.yourdomain.com	Public status page
hello@ support@ noreply@	Business email with SPF, DKIM and DMARC

- Replace the Vercel address in `index.html`, `sitemap.xml` and `robots.txt`
- Move the security headers from `vercel.json` to the new host; keep the content security policy in one place
- Share links currently use stitch.app: own that address or change the links
- Keep the old address as a redirect for a few weeks

7 Mobile App Requirements

Capacitor shell
Android and iPhone from the same code

Native push
Firebase on Android, Apple push on iPhone

Deep links
Stitch links open the app

Camera and media
Camera, microphone and photo picker

Device handling
Safe areas, keyboard, Android back button

Optional biometric lock
Fingerprint or face unlock

8 Store Requirements

Requirement	Apple App Store	Google Play
Developer account	About 99 USD per year	About 25 USD once; new accounts may need a closed test first
Sign in	Sign in with Apple required (Google login is offered)	Not required
Privacy	Privacy details form	Data safety form
Both stores	Privacy policy, terms, support page, in-app account deletion, reporting and blocking, reviewer account	Same

Key decision: payments inside the apps. Courses, class access and career subscriptions are digital products. Both stores require their own billing for digital content and take about 15% to 30%. Using Paystack in the apps for these may cause rejection. Options: (1) store billing in the apps and Paystack on the web, adjusting prices or the creator fee; (2) sell on the web only and let apps show owned content; (3) Paystack in apps only for real-world services after careful review. Decide before building the store versions.

9 Security Requirements

Area	Requirement
Database	Row-level security on every table, tested from the user's point of view
Money and permissions	Decided by the server: prices, fees, payouts, admin rights
Edge functions	JWT check on; only webhooks and test login verify callers another way
Files	Private buckets for IDs, selfies, CVs, documents, voice notes, chat media
Accounts	Two-step login for admins, domain, hosting, Supabase, Paystack, store accounts
Secrets	Backend only; rotate on a schedule and when someone leaves
Abuse	Rate limits on login, codes, messages, uploads, reports, AI
Privacy	Private screens hidden from analytics; no personal details in error reports
Testing	Outside penetration test before the main launch
Law	Register with Ghana's Data Protection Commission (Act 843); consider GDPR

10 Performance, Calls and Monitoring

Performance
Pagination everywhere, compressed images, streamed videos, split JavaScript, low-data mode

Calls
Replace free public relay with an owned or paid one; managed server for group calls; reconnect and audio-only fallback

Monitoring
Sentry and PostHog; alerts for failed payments, payouts, functions and jobs; public status page

Backups
Paid Supabase plan, daily backups, point-in-time recovery, tested restore, spend caps

11 Known Technical Debt

Item	Detail
Very large files	courses.js and jobs.js about 10,000 lines each, chat.js about 8,500, styles.css nearly 4,000
Old code	games.js still holds mini-game code and demo values
Leftover tables	Referral, game and old backup tables remain
JSON columns	Several tables keep most fields in one data column
Duplicate rules	Content security policy is in two places
Missing pieces	No web manifest; calls use free public relay
Fonts	Montserrat and Inter load from Google; Colmeak is already embedded

12 Open Technical Decisions

- Hosting provider (Cloudflare Pages recommended)
- Group call service

- Payment model in the store apps
- Whether to move the Supabase region closer to Ghana
- When to move to a build system and TypeScript
- Whether to rebuild the apps fully native later